Pràctiques de Sistemes Digitals i Microprocessadors

Curs 2025-2026

Pràctica 2

Fase B

LSFarm

	Login	Nom
Alumnes		

	Placa	Memòria	Nota
Entrega			

Data	

Còpia per als alumnes

Pràctiques de Sistemes Digitals i Microprocessadors

Curs 2025-2026

Pràctica 2

Fase B

LSFarm

	Login	Nom
Alumnes		

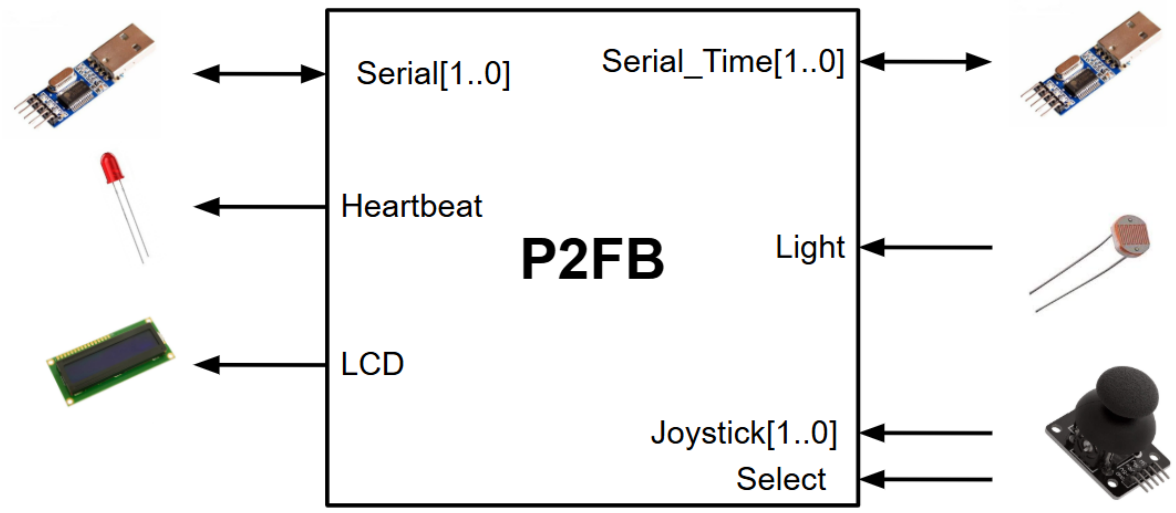
	Placa	Memòria	Nota
Entrega			

Data	

Portada de la memòria

1. Introducció i consideracions generals

En aquesta pràctica es proposa el disseny i la implementació, sobre un microcontrolador PIC18F4321 i mitjançant programació en C amb metodologia cooperativa, d'un sistema que simula el funcionament d'una granja. El sistema haurà de permetre inicialitzar la granja, generar animals i productes, gestionar el descans dels animals, mostrar informació a través d'un LCD i d'una interfície Java, persistir part de l'estat a la memòria EEPROM i respondre a les ordres de l'usuari mitjançant un canal sèrie.



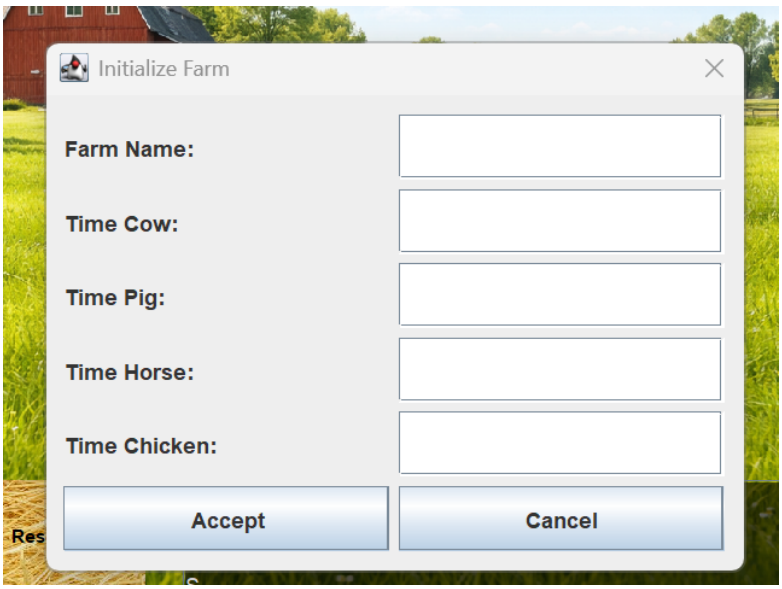
2. Descripció de les entrades i sortides del sistema

- `Heartbeat`: Indica que els animals no s'estan rebel·lant generant un PWM heartbeat.
- `Serial[1..0]`: Canal sèrie dedicat exclusivament a la comunicació amb la interfície Java, seguint el protocol especificat a l'Annex 2, assegurant la correcta transmissió i recepció de dades.
- `Serial_Time[1..0]`: Canal sèrie independent dedicat exclusivament a la introducció i actualització de la data i l'hora del sistema mitjançant terminal sèrie. Aquest canal no comparteix protocol amb la interfície Java.
- `Light`: Senyal analògica que indica quanta llum ambiental hi ha.
- `Joystick amb polsador`: S'utilitza per navegar pel menú de la interfície Java.
- `LCD`: Serveix per mostrar informació del sistema a l'usuari, com el nom de la granja, la data i notificacions d'esdeveniments.

3. Funcionament del Sistema

Quan el sistema s'encén, queda a l'espera de rebre, d'una banda, la configuració enviada des de la interfície Java (`Serial[1..0]`) i, de l'altra, la data i l'hora inicials introduïdes per `Serial_Time[1..0]`. Fins que no s'hagin rebut correctament tots aquests paràmetres, el sistema no entrarà en funcionament normal.
Els paràmetres que es podran configurar des de la interfície Java i que s'enviaran són:

- Nom de la granja
- Temps de generació d'una espècie: s'especificaran 4 temps de generació en unitats de segon, un per cada espècie.



Il·lustració 1: Foto Java inicialitzar

D'altra banda, l'usuari haurà d'introduir pel canal sèrie `Serial_Time` la data i temps inicials en format `DD/MM HH:MM:SS`. La data i l'hora introduïdes s'hauran de poder visualitzar per la terminal.

```
27/01 12:21:03
Date and time correct
72/01 12:21:03
Please input a correct date
```

Il·lustració 2: Inicialització data

Un cop configurats els paràmetres anteriors, el sistema entra en funcionament. Durant l'execució, l'usuari podrà actualitzar la data i hora actuals introduint-ho pel canal sèrie en el mateix format que en la inicialització.

Un cop inicialitzat el sistema, els animals de cada espècie es generaran periòdicament segons el temps configurat per a aquella espècie. S'hauran de poder guardar un mínim de 3 animals per espècie i un màxim de 24 animals en total.

Un cop creats, cada animal generarà un tipus de producte concret cada cert temps depenent de la espècie. Les gallines produiran ous cada 13 segons, els cavalls pinzells cada 23 segons, els porcs pernil cada 31, i les vaques llet cada 47 segons. No obstant, per seguir produint, els animals necessiten descansar. Cada 2 min, els animals entraran en estat de son crític. Un cop entrin en aquest estat, romandran a l'espera de que l'usuari els seleccioni per descansar. Mentre esperen, no generaran res.

Llavors, cada vegada que finalitzi el temps d'una espècie, tots els animals d'aquella espècie que existeixin en aquell instant i que no es trobin en estat de son crític produiran simultàniament una unitat del producte corresponent.

Per mantenir al granger informat dels esdeveniments de la granja, s'utilitzarà un LCD. A l'inici, i sempre que no hi ha hagi res a notificar, el LCD mostrarà el nom de la granja i la data actual, amb el següent missatge:

LSFarm-COMPUS! 13/03/2026

Il·lustració 3: Indica com s'ha de veure el missatge del LCD quan no hi ha notificacions

Quan es generi un animal, el LCD haurà de notificar-ho especificant el tipus i el número. De la mateixa manera, quan es generi un producte, s'haurà de mostrar a traves del display el número total i el tipus de producte que s'ha generat. Les notificacions hauran de ser visibles un mínim de 3 segons, seguint els corresponents exemples.

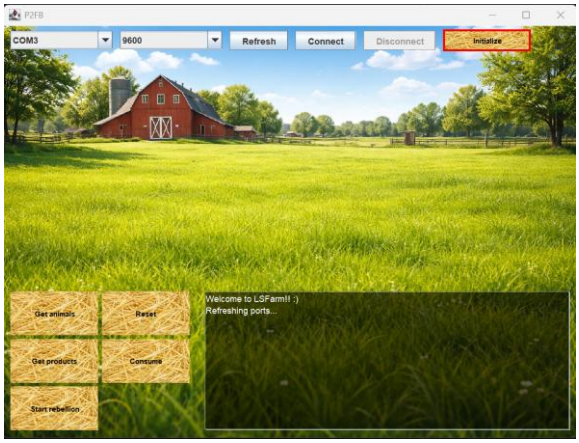
Nou Producte Ous: 11

Nou Animal Vaca: 2

Il·lustració 4: Indica com s'ha de veure el missatge del LCD quan hi ha una notificació

L'estat individual dels animals, així com el número d'animals de cada espècie, ha de persistir en el temps. Per cada animal, s'haurà de guardar el seu tipus i l'últim instant de temps (dia, mes, hora, minut i segon) en el que han dormit a la memòria EEPROM interna del microcontrolador.

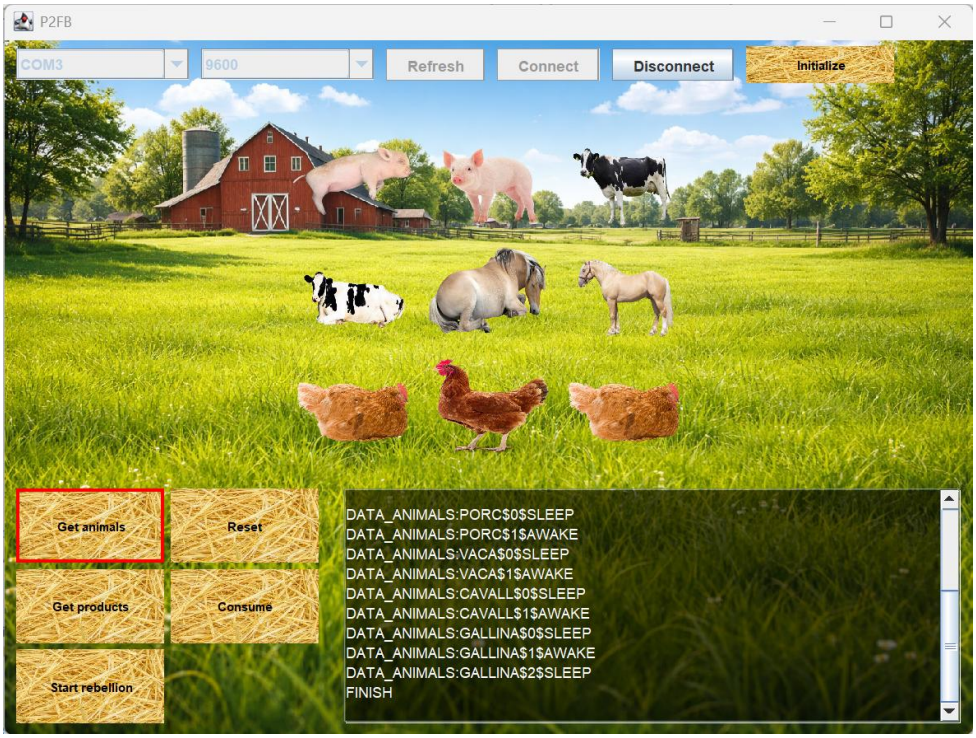
Per interactuar amb el microcontrolador, es farà us de una interfície Java. Per moure's en el menú d'aquesta interfície s'utilitzaran els dos eixos del Joystick i per confirmar la selecció d'una opció s'utilitzarà el botó que aquest incorpora. El menú constarà de 5 funcionalitats: reset del sistema, visualització dels productes, visualització dels animals, àrea de consum i rebel·lió dels animals. Referiu-vos a l'Annex 2 per veure el protocol a realitzar per poder comunicar el microcontrolador i la interfície Java.



Il·lustració 5: Interfície Java

Visualització dels animals

Aquesta funcionalitat permetrà veure els diferents animals que es troben a la granja i el seu estat de son. L'usuari es podrà moure pels diferents animals utilitzant també el joystick. Un cop identifiquin un animal que necessiti descansar, podran seleccionar-lo també amb el botó del joystick. Una vegada s'hagi seleccionat l'animal, l'usuari disposarà de 5 segons per tapar el sensor LDR permetent que l'animal descansi. Si es detecta correctament la manca de llum, s'enviarà un missatge a la interfície perquè marqui l'animal com a descansat i es guardarà a la EEPROM l'hora, minut i segon actuals, actualitzant l'última vegada on l'animal ha dormit. Altrament, el microcontrolador enviarà un missatge indicant que l'animal no ha pogut descansar.



Il·lustració 6: Resultat de Get animals

Visualització dels productes

La funcionalitat permetrà visualitzar la quantitat de productes de cada tipus dels que la granja disposa actualment en format de gràfica. El microcontrolador haurà d'enviar el número actual de productes en el format especificat a l'Annex 2.

Il·lustració 7: Resultat de Get products

Reset del sistema

Quan s'executi aquesta opció l'estat de la granja serà restablert. Això implicarà l'eliminació de totes les dades emmagatzemades a la EEPROM. La granja quedarà buida de animals i productes i el sistema romandrà a l'espera de la inicialització.

Rebel·lió animals

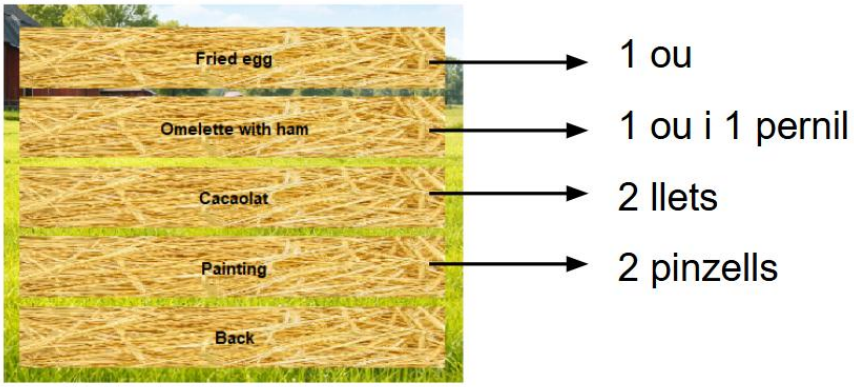
Aquesta funcionalitat permetrà a l'usuari poder parar i restablir la producció dels animals. Mentre aquesta opció estigui activada, els animals deixaran de produir temporalment fins que l'usuari torni a restablir el funcionament del sistema. Mentre duri la rebel·lió, el LED Heartbeat s'apagarà. En condicions de funcionament normal, generarà un PWM tipus Heartbeat.

Àrea de consum

Aquesta funcionalitat permetrà al usuari escollir diverses opcions que consumiran un cert nombre de productes. Per exemple, la opció de fer-se un truita amb pernil. Això es comunicarà al microcontrolador, que actualitzarà la quantitat de productes de manera que el consum es vegi reflectit a la gràfica.

Il·lustració 8: Apartat àrea de consum

En concret, es podran consumir els diferents productes i consumiran la següent quantitat de productes:



Il·lustració 9: Consumició per producte

Consideracions generals

1. Tota la lògica específica de l'enunciat s'ha de resoldre exclusivament a la banda del microcontrolador.
2. El microcontrolador ha d'estar programat exclusivament en C mitjançant la metodologia de sistemes cooperatius.
3. El polsador del Joystick podrà ser substituït per un polsador separat.
4. Si hi ha solapament de notificacions al LCD, s'hauran de poder acumular un mínim de 3. Es valorarà positivament que se'n puguin acumular més.
5. El temps de generació per cada tipus d'animal enviat durant la fase de configuració podrà prendre valors d'entre 1 i 60 segons.
6. Es pot assumir que el sistema sempre es trobarà en l'any actual.
7. La pràctica està especialment dissenyada per fer-vos reflexionar sobre l'ús de memòria del microcontrolador. Enteneu que aquest és un recurs car i escàs; poseu especial èmfasi en el disseny de les estructures de dades i el vostre software.
8. La memòria ha de contenir els apartats indicats en l'Annex 1.
9. Al lliurament d'aquesta fase cal entregar un fitxer .zip que contingui una memòria que compleixi la normativa de pràctiques, un vídeo o enllaç a vídeo de la pràctica funcionant en la seva totalitat, fotografies de la placa, i una carpeta amb tot el projecte i el resultat de la seva compilació (fitxers .c, .h, i .hex). Cal afegir l'arxiu dictionary.c, tal i com s'ha ensenyat a les classes teòriques de l'assignatura. El nom del fitxer .zip ha de seguir el següent format:

YYYY.MM.DD-SDM-2526-P2-FB-login1-login2.zip
Per Exemple: 2026.03.15-SDM-2526-P2-FB-john.doe-jane.doe.zip

Annex 1: Seccions que ha de tenir la memòria

Cal que la memòria de la pràctica contingui els següents apartats:

- **Portada i índex**: La portada ha de ser la d'aquest mateix document, i cal que a l'índex les pàgines estiguin numerades.
- **Resum de l'enunciat**: Amb les vostres paraules, cal que detalleu què ha de fer la pràctica.
- **Plantejament del software**: Explicació de com estructurareu el software del microcontrolador.
- **Configuracions del microcontrolador**: Explicació i justificació de les configuracions utilitzades, ja siguin a nivell de pins, ports, registres, interrupcions, timers...
- **Diagrama de TADs**: Explicat i detallat amb els perifèrics i variables més importants.
- **Diccionari.**
- **Motors dels TADs**: Captura de cada motor amb la seva corresponent explicació.
- **Esquema elèctric**.
- **Conclusions i problemes observats**.
- **Planificació**: Ha de contenir dos diagrames de Gantt. El primer ha de fer referència a la planificació inicial de la pràctica, i el segon ha de mostrar els timings reals després de la resolució final de la pràctica. S'ha d'explicar les variacions entre ambdós diagrames.

Annex 2: Protocol de comunicació

Bàsicament explicar com tots els missatges que poden haver entre PIC—JAVA.

Aquest annex descriu el protocol configurable que s'utilitza per gestionar la comunicació sèrie entre la aplicació JavaFX i un microcontrolador PIC. El protocol està definit a partir d'un conjunt de constants agrupades en una classe específica anomenada FarmConstants. Per poder implementar el sistema, caldrà modificar aquesta classe amb l'objectiu de definir i adaptar un protocol de transmissió propi i personalitzat.

1. Comunicació Java → PIC

L'aplicació Java envia ordres específiques al PIC mitjançant cadenes ASCII acabades en retorn de carro i salt de línia (\r\n). El protocol d'aquestes ordres s'ha de configurar modificant els seus valors a la classe FarmConstants.

Al llarg del funcionament del sistema, la interfície Java enviarà o sol·licitarà dades al PIC a través de comunicacions sèrie que han de seguir un format prèviament establert. En els exemples següents es mostren les constants que caldrà adaptar, juntament amb la informació addicional necessària en cada transmissió, per tal de poder desenvolupar correctament les diferents funcionalitats del sistema.

<u>Inicialitzar Granja</u>

- FarmConstants.CMD_INITIALIZE + "nomGranja$TempsCreacióVaca$TempsCreacióCavall$TempsCreacióPorc$TempsCreacióGallina\r\n"

<u>Sol·licitar animals</u>

- FarmConstants.CMD_GET_ANIMALS + "\r\n"

<u>Sol·licitar productes</u>

- FarmConstants.CMD_GET_PRODUCTS + "\r\n"

<u>Consumir productes</u>

- FarmConstants.CMD_CONSUME + "OptionNum\r\n"

<u>Reiniciar Sistema</u>

- FarmConstants.CMD_RESET + "\r\n"

<u>Rebel·lió animals</u>

- FarmConstants.CMD_START_REBELLION + "\r\n"
- FarmConstants.CMD_STOP_REBELLION + "\r\n"

Dormir animal

- FarmConstants.CMD_SLEEP + "TipusAnimal$NumeroAnimal\r\n"
- Exemples:
    - FarmConstants.CMD_SLEEP + "VACA$1\r\n"
    - FarmConstants.CMD_SLEEP + "CAVALL$1\r\n"
    - FarmConstants.CMD_SLEEP + "GALLINA$1\r\n"
    - FarmConstants.CMD_SLEEP + "PORC$1\r\n"

2. Comunicació PIC → Java

El PIC transmet ordres o dona resposta a les peticions de Java mitjançant cadenes en format ASCII que finalitzen amb retorn de carro i salt de línia (\r\n). El format d'aquestes comandes s'ha d'ajustar modificant els valors corresponents dins la classe FarmConstants.

Resposta a GET PRODUCTS

- FarmConstants.CMD_DATA_PRODUCTS + "NumLlet$NumPernil$NumOus$NumPinzells\r\n"

Resposta a SLEEP

- FarmConstants.CMD_SLEEP_SUCCESSFUL + "\r\n"
- FarmConstants.CMD_SLEEP_UNSUCCESSFUL + "\r\n"

Resposta a GET ANIMALS

- Dades enviades amb el format: FarmConstants.CMD_DATA_ANIMAL + "TipusAnimal$Numero$Estat\r\n"

- Exemples:

    - CMD_DATA_ANIMAL + "VACA1SLEEP\r\n"

    - CMD_DATA_ANIMAL + "CAVALL1AWAKE\r\n"

    - CMD_DATA_ANIMAL + "PORC1AWAKE\r\n"

    - CMD_DATA_ANIMAL + "GALLINA1SLEEP\r\n"

- Comanda de finalització: FarmConstants.CMD_FINISH+"\r\n"

Moure amunt

- FarmConstants.CMD_UP + "\r\n"

Moure avall

- FarmConstants.CMD_DOWN + "\r\n"

Moure a la esquerra

- FarmConstants.CMD_LEFT + "\r\n"

Moure a la dreta

- FarmConstants.CMD_RIGHT + "\r\n"

Seleccionar opció

- FarmConstants.CMD_SELECT + "\r\n"

Exemples d'interaccions

Sol·licitar productes

1. Java->PIC
   FarmConstants.CMD_GET_PRODUCTS + "\r\n"
2. PIC -> Java
   FarmConstants.CMD_DATA_PRODUCTS + "NumLlet$NumPernil$NumOus$NumPinzells\r\n"

Sol·licitar animals

1. Java->PIC
   FarmConstants.CMD_GET_ANIMALS + "\r\n"
2. PIC->Java (per cada animal)
   FarmConstants.CMD_DATA_ANIMAL + "VACA1SLEEP\r\n"
3. PIC-> Java
   FarmConstants.CMD_FINISH + "\r\n"

Dormir animal

1. Java->PIC
   FarmConstants.CMD_SLEEP + "VACA$1 \r\n"
2. PIC -> Java (Si l'animal aconsegueix dormir en menys de 5 segons)
   FarmConstants.CMD_SLEEP_SUCCESSFUL + "\r\n"
3. PIC -> Java (Si no aconsegueix dormir)
   FarmConstants.CMD_SLEEP_UNSUCCESSFUL + "\r\n"